

Rupture de symétrie (lex-leader) – de A à Z

1 Le problème de départ

Une molécule benzénoïde donnée (un graphe dual G_D , un sommet par cycle) peut avoir des **symétries géométriques** – rotations, réflexions. Si on substitue des tailles 5/6/7 aux sommets sans tenir compte de ça, le solveur va trouver, pour une substitution donnée, **toutes ses copies symétriques** comme des solutions distinctes – alors que c’est chimiquement la **même molécule**. Sans correction, on compterait (et énumérerait) des doublons inutiles.

2 Étape 1 – détecter les automorphismes (bibliothèque)

Un **automorphisme** de G_D est une permutation π des sommets qui **préserve toutes les arêtes** : si i, j sont voisins, alors $\pi(i), \pi(j)$ doivent aussi être voisins. C’est exactement la traduction graphique d’une symétrie géométrique de la molécule – pas besoin de connaître la géométrie, l’automorphisme du graphe suffit.

Concrètement, dans `preprocessing.py::compute_automorphisms` :

```
G = graph.dual
matcher = nx.algorithms.isomorphism.GraphMatcher(G, G)
for iso in matcher.isomorphisms_iter():
    ...
```

On compare G_D à **lui-même** avec l’algorithme **VF2** (implémenté dans NetworkX) : il essaie, sommet par sommet, toutes les correspondances bijectives qui respectent les arêtes, et les énumère toutes. On exclut ensuite l’identité (la permutation qui ne bouge rien), qui n’apporte aucune information de symétrie.

À ce stade, on a une liste potentiellement longue : **tous** les automorphismes non triviaux de G_D .

3 Étape 2 – réduire à un ensemble générateur minimal

Poser une contrainte pour *chaque* automorphisme trouvé serait redondant : si π_1 et π_2 sont déjà connus, leur composée $\pi_1 \circ \pi_2$ est automatiquement un automorphisme aussi, pas besoin de la traiter séparément. On cherche donc un sous-ensemble minimal qui **engendre** tout le reste par composition.

L’algorithme (glouton), implémenté dans `preprocessing.py::_extract_generators` :

```
generateurs <- [] ; groupe <- { identite }
pour chaque automorphisme a trouve (hors identite) :
    si a n’est pas déjà dans groupe :
        generateurs <- generateurs + [a]
        groupe <- cloture(generateurs) # toutes les compositions possibles
        si groupe == tous les automorphismes trouves : arreter
retourner generateurs
```

La « clôture » (`_generate_group`) compose itérativement les générateurs entre eux jusqu’à ce qu’aucune nouvelle permutation n’apparaisse (point fixe). On s’arrête dès que cette clôture couvre tout ce qu’on a trouvé à l’étape 1 – donc on ne garde que le strict nécessaire, $\{\pi_1, \dots, \pi_m\}$, avec m généralement petit (souvent 1 ou 2 pour ces graphes) même si le groupe complet peut être grand.

Nuance honnête : c’est une heuristique simple, pas l’algorithme le plus rigoureux pour de très gros groupes (la référence serait Schreier–Sims) – mais suffisant ici.

4 Étape 3 – poser la contrainte

Dans `model.py` :

```
for gen in generators:
    permuted = [x[gen[i]] for i in range(h)]
    satisfy(LexIncreasing(x, permuted))
```

Pour chaque générateur π , on construit le vecteur « permuté » $(x_{\pi(0)}, \dots, x_{\pi(h-1)})$ et on impose :

$$(x_0, \dots, x_{h-1}) \leq_{\text{lex}} (x_{\pi(0)}, \dots, x_{\pi(h-1)})$$

`LexIncreasing` est une **contrainte globale** standard (XCSP3/Choco) : sa portée $S(c)$ est le vecteur entier, sa relation $R(c)$ l'ensemble des tuples qui respectent cet ordre. On ne code pas l'algorithme qui la fait respecter pendant la recherche – ça, c'est interne au solveur – on se contente de la déclarer.

Effet : parmi toutes les solutions équivalentes par symétrie (une orbite entière), seule celle qui est lexicographiquement minimale satisfait *simultanément* toutes ces inégalités. Les autres sont rejetées par le solveur – elles ne sont même pas énumérées.

5 Le résidu imparfait (et son rattrapage)

En pratique, Choco propage `LexIncreasing` un peu moins strictement qu'ACE (l'autre solveur utilisé) : il laisse parfois passer $\sim 0,26\%$ de solutions qui sont en fait dans la même orbite qu'une autre déjà acceptée. Un post-filtre Python (`model.py::_dedup_by_orbit`) rattrape ça après coup : pour chaque solution renvoyée, il recalcule toute son orbite (en appliquant les générateurs récursivement) et ne garde que le représentant minimal – exactement le même critère que la contrainte, mais vérifié une seconde fois côté Python pour garantir l'alignement exact entre les deux solveurs.

En une phrase

Détecter (bibliothèque) $\rightarrow$ réduire à un générateur minimal (code du projet) $\rightarrow$ poser une inégalité lex par générateur (déclaratif, contrainte globale) $\rightarrow$ laisser le solveur filtrer pendant la recherche $\rightarrow$ re-vérifier après coup pour combler l'imperfection de la propagation.